

Dez ferramentas, um só teste: *como comparar frameworks sem cair no “top 5 melhores”*

DOUGLAS QUEIROZ · INSTRUTOR · QA / AUTOMAÇÕES

- **A pergunta que abre a trilha**

“Qual é o melhor framework de teste?”

Essa é a pergunta que mais chega para mim, e a resposta honesta — “depende” — não ajuda ninguém. O problema não está na pergunta. Está em como ela costuma ser respondida: quase todo comparativo publicado mede cada ferramenta resolvendo um problema diferente, escrito por quem já usa uma delas, e sem dizer uma palavra sobre o custo de manter aquilo daqui a um ano.

Falta o que qualquer experiência precisa ter: **variável controlada**. Se o cenário muda junto com a ferramenta, não há comparação — há preferência com aparência de análise.

Esta trilha nasceu porque eu encontrei o material que resolve isso. Um acervo público no GitHub com vinte repositórios de teste, criados pela mesma equipe, no mesmo período, cada um dedicado a uma ferramenta — e, em boa parte deles, implementando **exatamente o mesmo caso de teste**.

- **O material: vinte repositórios, um cenário**

São 20 repositórios com prefixo `qake-*`, criados entre setembro de 2022 e maio de 2023. Dezesete têm código utilizável; dois são placeholder e um está completamente vazio. Dez atacam teste de API, três teste de componente, um E2E com autenticação e três contrato e virtualização de serviço. Nove linguagens e DSLs no total.

O detalhe que transforma isso em bancada de comparação: em **seis dos dez repositórios de API** o alvo é idêntico — a mesma API pública, os mesmos três verbos, as mesmas asserções. Abaixo, o mesmo caso escrito nos dois extremos do espectro.

```
# Hurl - HTTP em texto puro
GET https://jsonplaceholder.typicode.com/comments?id=4
HTTP/1.1 200
[Asserts]
jsonpath "$[0].postId" == 1

# Karate - a mesma coisa em Gherkin
Given path 'comments'
And param id = 4
When method get
Then status 200
And match response == read('resources/comments.json')
```

Mesmo raciocínio, duas experiências de escrita completamente diferentes — e ainda existem outras oito formas de escrever isso no acervo, de Playwright a Robot Framework, de Go a Java. É esse contraste,

repetido dez vezes sobre o mesmo alvo, que a trilha vai dissecar.

Liga com a técnica: isto é exatamente o que o capítulo 6 do syllabus CTFL 4.0 trata — ferramentas de suporte ao teste, os benefícios e os **riscos** da automação. A seção 6.2.1 fala de expectativas irreais e de subestimar o esforço de manutenção; a 6.1.1 lembra que ferramenta apoia atividade de teste, não substitui pensamento. Cada episódio desta trilha é um estudo de caso desses dois pontos.

• O método: quatro coisas em todo episódio

- **Tempo real de setup.** Do repositório clonado ao primeiro teste verde, cronometrado. É o número que decide adoção em time real e o que nenhum comparativo publica.
- **O código na íntegra.** Não o trecho bonito do README: o arquivo de teste inteiro, com o que ele importa, o que repete e o que esconde.
- **A mensagem de erro.** O que a ferramenta devolve quando o teste falha. É o texto que você vai ler todo dia — e o que mais diferencia framework na prática.
- **O custo de manter.** Dependências, versão travada, quem no time consegue mexer e o que acontece quando a pessoa que escreveu a suíte sai.

• Quatro achados que já garantiram episódio

A leitura do código rendeu mais que comparação de sintaxes. Quatro problemas apareceram e cada um vale uma aula inteira, porque é o tipo de coisa que qualquer um de nós já entregou:

- **Dois testes que nunca rodam.** Os arquivos não terminam em `.test.js` e o glob do script não os enxerga. Estão versionados, e verdes por omissão.
- **Um README que descreve outra stack.** Anuncia Jest e SuperTest; o código é Cypress com AJV. A primeira linha do repositório já está errada.
- **Um repositório “pact-test” sem Pact.** Nenhum pact file, nenhum broker, nenhuma biblioteca — a ferramenta ali é Specmatic, que é outra escola de teste de contrato.
- **Dezoito de vinte sem CI.** O argumento central da automação é rodar sem gente olhando. Quase nenhum exemplo do acervo roda sozinho.

O que isso ensina: o primeiro caso é o melhor argumento que eu conheço para uma regra simples — **confie no relatório de execução, não na pasta de testes**. Suíte verde com zero teste executado é verde igual. É o mesmo ponto cego que a aula de cobertura já mostrou por outro caminho.

• O trabalho no código: forks mantidos e o mesmo alvo

A trilha não termina no texto. Em paralelo aos episódios, cada repositório analisado vira um **fork mantido**: o que está quebrado é corrigido, as versões saem do congelamento de 2023 e entram as coisas que faltam em quase todo o acervo — pipeline de CI, licença, descrição, `.gitignore` e a remoção dos artefatos de execução versionados.

A segunda frente é a que torna o comparativo realmente justo: **padronizar o alvo**. Hoje quatro dos dez repositórios de API apontam para alvos distintos, e os testes de interface e de componente rodam cada um sobre uma aplicação diferente. O trabalho é levar todos ao mesmo padrão:

- **Uma API de referência** para todo teste de API, com os mesmos endpoints e os mesmos casos em todas as ferramentas.

- **Um site de referência** para todo teste de interface e de componente, com os mesmos fluxos exercitados em todas elas.
- **As mesmas verificações dos dois lados**, para que a comparação entre teste de interface e teste de API deixe de ser retórica e passe a ser medida.

Essa última linha é o ponto mais interessante e o menos coberto na literatura. Comparar Playwright com Robot Framework é comum; comparar o custo de verificar *a mesma regra* pela API e pela tela — tempo de execução, fragilidade, o que cada caminho enxerga e o que deixa passar — quase nunca é feito com o mesmo alvo dos dois lados. Com o acervo padronizado, passa a ser.

Quer participar? Os forks ficam abertos. Dá para pegar um repositório da trilha e assumir uma tarefa pequena e real: adicionar o workflow de CI, corrigir o glob que não pega os testes, atualizar uma versão, apontar a suíte para a API de referência. É experiência de manutenção de suíte em projeto público — o tipo de coisa que rende conversa em entrevista. Fale comigo (QA / Automações).

- **A trilha, episódio por episódio**

BLOCO 1 · TESTES DE API

- 1 **Robot Framework** — legibilidade tem preço?
- 2 **Playwright para API** — vale usar o canhão do E2E?
- 3 **Karate** — o payload inteiro validado em uma linha
- 4 **Godog e Gherkin em Go** — quando BDD agrega e quando é só camada
- 5 **Hurl** — quanto framework você realmente precisa?
- 6 **Jest + SuperTest** — o uso idiomático que quase ninguém faz
- 7 **TestCafe** — o teste que existe e nunca roda
- 8 **JUnit + REST Assured + Allure** — a stack corporativa e o peso do setup
- 9 **RSpec** — testar a API que é sua
- 10 **Cypress na nuvem** — grid, paralelismo e o segredo que ficou no commit

BLOCO 2 · TESTE DE COMPONENTE

- 11 **jsdom contra navegador real** — Jest + RTL e Cypress Component lado a lado
- 12 **Componente assíncrono** — mockar rede sem mockar demais

BLOCO 3 · E2E E AUTENTICAÇÃO

- 13 **Login em E2E** — pela tela, por API ou por sessão

BLOCO 4 · CONTRATO E VIRTUALIZAÇÃO

- 14 **Schema não é contrato** — onde o AJV para
- 15 **Pact contra Specmatic** — as duas escolas de teste de contrato
- 16 **WireMock e stub** — três formas de isolar o mundo lá fora

- **Nota final: o que este texto ainda não fez**

Coerência exige o registro: **nenhum teste foi executado até aqui**. Tudo o que está escrito acima vem da leitura do código, da configuração e dos manifests de dependência. A execução — com tempo cronometrado e mensagem de erro capturada — é justamente o que cada episódio traz, e é por isso que a trilha tem episódios em vez de um artigo só.

Sempre que o README e o manifest divergiram, tratei o manifest como fonte da verdade e anotei a divergência. Isso aconteceu em pelo menos quatro repositórios — e virou material de aula, não nota de rodapé.

Uma última observação, e ela importa: os problemas apontados aqui não são crítica a quem escreveu esses repositórios. São exemplos didáticos que alguém teve o trabalho de publicar em aberto, e é por existirem em aberto que esta trilha é possível. Leitura crítica é o que se faz com material bom.

“Framework não se escolhe por opinião. Se escolhe medindo o mesmo problema, com o mesmo alvo, e olhando o custo de manter.”